

With the help of Prompt engineering concept and construct do the following. Design and development of Chat Bot using Generative AI tools to be embedded in website created by MERN technologies for online stock learning institute

**Submitted in partial fulfilment of the requirements of the degree
Bachelor of Engineering in Artificial Intelligence and Data Science**

By

Aryan Bhagat – 2301011

Shahoo Birwadkar – 2301014

Pratik Jadhav – 2301044

Mohd Arman Iddrishi – 2301041

Supervisor

Dr. G. T. Thampi



Department of Artificial Intelligence and Data Science

Thadomal Shahani Engineering College

Bandra (W) Mumbai – 400 050

University of Mumbai

(A.Y 2023 – 24)

CERTIFICATE

This is to certify that the Mini Project entitled “**With the help of Prompt engineering concept and construct do the following. Design and development of Chat Bot using Generative AI tools to be embedded in website created by MERN technologies for online stock learning institute**” is a bonafide work of **Aryan Bhagat (Roll no. 2301011)**, **Shahoo Birwadkar (Roll no. 2301014)**, **Pratik Jadhav (Roll no. 2301044)** and **Mohd Arman Iddrishi (Roll no. 2301041)** submitted to the University of Mumbai in partial fulfilment of the requirement for the award of the degree of “**Bachelor of Engineering**” in “**Artificial Intelligence and Data Science**”.

Dr. G. T. Thampi

Supervisor

(Dr. Madhuri Rao)

Head of Department

(Dr. G. T. Thampi)

Principal

Mini Project Approval

This Mini Project entitled “**With the help of Prompt engineering concept and construct do the following. Design and development of Chat Bot using Generative AI tools to be embedded in website created by MERN technologies for online stock learning institute**” by **Aryan Bhagat (Roll no. 2301011), Shahoo Birwadkar (Roll no. 2301014), Pratik Jadhav (Roll no. 2301044) and Mohd Arman Iddrishi (Roll no. 2301041)** is approved for the degree of “**Bachelor of Engineering**” in “**Artificial Intelligence and Data Science**”.

Examiners

1.

(Internal Examiner Name & Sign)

2.

(External Examiner Name & Sign)

Date:

Place:

CONTENTS

Abstract	1
Acknowledgments	2
List of Abbreviations	3
List of Figures	4
1 Introduction	5
1.1 Introduction	5
1.2 Motivation	5
1.3 Problem Statement & Objectives	6
1.4 Organization of the Report	7
2 Literature Survey	8
2.1 Survey of Existing System	8
2.2 Limitation Existing System or Research Gap	8
2.3 Mini Project Contribution	9
3 Proposed System	10
3.1 Introduction	10
3.2 Architecture / Framework	11
3.3 Algorithm and Process Design	14
3.4 Details of Hardware & Software	17
3.5 Experiment and Results	18
3.6 Conclusion	20
References	21

Abstract

The advancement of digital technology has ushered in a new era of innovation, particularly in online education and web-based learning. To enhance user experience on a website developed for an online stock learning institute, this project proposes a structured approach to leverage **Generative Artificial Intelligence (AI)** with **Prompt Engineering** for chatbot design and development. The chatbot acts as an intelligent virtual tutor, helping learners understand stock market concepts, investment terms, and trading strategies interactively.

The solution is designed using the **MERN (MongoDB, Express.js, React.js, Node.js)** stack and begins with thorough requirements analysis and architectural planning. It includes essential modules such as a conversational model, prompt engineering framework, data integration layer, reporting tools, and a user-friendly interface. Each part is carefully developed to ensure scalability, maintainability, and smooth integration with the online learning platform.

Prompt engineering plays a vital role by shaping how the AI interprets and responds to users. Through well-crafted prompts, the chatbot can understand intent, provide accurate explanations, and adapt to different learner levels—beginner, intermediate, or advanced—ensuring personalized learning. This approach enhances response quality and reduces irrelevant or inaccurate answers.

The chatbot undergoes rigorous testing for accuracy, adaptability, and engagement. It integrates seamlessly with the MERN-based website, offering instant support within the learning dashboard. The system is optimized for real-time interaction and can access course databases for context-aware responses.

To improve continuously, the chatbot learns from user interactions and feedback, refining its performance using machine learning methods. By using large datasets related to finance and trading, it can deliver more accurate, data-driven educational insights. Integration with financial APIs and databases enables it to present real-time stock examples, chart analyses, and simulated trading exercises, bridging theory and practice.

Aligned with the website's diverse functions, the chatbot connects easily with tools like learning management systems for progress tracking, content management systems for course updates, and analytical dashboards for performance reports. It supports various content formats—text, charts, PDFs, and videos—catering to different learning styles.

Overall, this AI-powered chatbot, designed using **Prompt Engineering** and developed with **MERN technologies**, represents a significant innovation in creating intelligent, adaptive, and interactive platforms for stock market education

Acknowledgement

We would like to express our gratitude and thanks to **Dr. G. T. Thampi** for his valuable guidance and help. We are indebted for his guidance and constant supervision as well as for providing necessary information regarding the project. We would like to express our greatest appreciation to our principal **Dr. G.T. Thampi** and head of the department **Dr. Madhuri Rao** for their encouragement and tremendous support. We take this opportunity to express our gratitude to the people who have been instrumental in the successful completion of the project.

Aryan Bhagat – 2301011

Shahoo Birwadkar – 2301014

Pratik Jadhav – 2301044

Mohd Arman Iddrishi – 2301041

Introduction

1.1 Introduction

The evolution of online learning platforms has experienced a revolutionary impact, particularly in the age of artificial intelligence (AI). The main idea behind this project is to employ Generative AI and Prompt Engineering to design and develop an intelligent chatbot aimed at enhancing user experience on an online stock learning platform built using MERN technologies. It explores the integration of AI-driven conversational agents within the educational web platform to make learning more interactive, personalized, and efficient.

1.2 Motivation

Improving the Learning Experience:

In the world of stock market education, learner engagement and clarity are crucial. A generative AI chatbot can provide personalized guidance, explain complex stock market concepts, offer real-time examples, and assist users with course navigation. This intelligent interaction can significantly enhance the overall learning experience, making the process more adaptive and enjoyable for students.

Enhancing Website Functionality:

For a dynamic online learning website, performance and responsiveness are essential. The integration of a Generative AI chatbot helps optimize user interactions by providing instant query resolution, course recommendations, and interactive tutorials. This seamless integration improves website usability and ensures that learners can access educational resources efficiently without delays.

Empowering Learners and Educators:

Students and instructors both benefit from an AI-enabled platform that supports communication and collaboration. Acting as a virtual mentor, the chatbot can assist learners with doubt-solving, provide quiz-based feedback, and track learning progress. For educators, it offers analytical insights into student performance and engagement trends. By connecting learners and instructors effectively, the platform becomes a comprehensive hub for stock market education and continuous learning.

Fig. 1.2.1

1.3 Problem Statement & Objectives

“With the help of Prompt engineering concept and construct do the following. Design and development of Chat Bot using Generative AI tools to be embedded in website created by MERN technologies for online stock learning institute.”

Creating a Generative AI chatbot for an online stock learning platform involves a comprehensive approach that combines artificial intelligence, web development, and domain-specific knowledge of the stock market. The project focuses on integrating Generative AI and Prompt Engineering techniques within a MERN-based website to enhance learning engagement and user interaction. The chatbot will serve as an intelligent virtual tutor capable of providing explanations, course guidance, and stock market insights in a conversational and personalized manner.

Basic Functions of MERN Stack:

The project will utilize the MERN (MongoDB, Express.js, React.js, Node.js) technology stack along with AI frameworks and APIs. MongoDB will manage user data and chat history, Express.js and Node.js will handle the server-side logic and API communication, while React.js will be used to design an intuitive and interactive frontend interface. Together, these technologies will enable the development of a dynamic and intelligent stock learning website.



Fig. 1.3.1

Objectives:

- **Enhancing the Learning Experience:**

The primary objective is to design an AI-powered chatbot that delivers an interactive and user-friendly interface. It should engage learners through natural, human-like conversations, explain stock market concepts clearly, and guide them through courses and practice exercises in real time.

- **Improving Educational Efficiency:**

The Generative AI chatbot aims to simplify complex financial topics by generating adaptive responses and personalized study materials. It will assist students with course-related queries, provide stock examples, and recommend learning paths based on individual progress.

- **Seamless Integration and Performance Optimization:**

Another key objective is to ensure smooth integration of the chatbot within the MERN-based website. The chatbot should operate efficiently without affecting website performance, ensuring fast response times and a consistent learning experience.

- **Supporting Instructors and Learners:**

The chatbot will also act as a digital assistant for instructors by gathering student feedback, tracking learner engagement, and generating reports. This will help educators monitor progress and improve teaching effectiveness, making the platform a comprehensive AI-driven learning environment.

1.4 Organization of the Report

This report consists of three chapters. The first chapter deals with introduction of the topic, problem statement, motivation behind the topic and objectives. The second chapter is the Literature Survey. It includes all the research work done related to this topic. All information related to study of existing systems as well as learning of new tools is mentioned in this chapter. The third chapter is about the proposed system which is used in this project. The block diagram, techniques used, hardware and software used screenshots of the project are presented in this chapter.

All the documents related to development of this project are mentioned in References.

Literature Survey

2.1 Survey of Existing System

1. LangChain:

LangChain is a powerful open-source framework for developing applications using Large Language Models (LLMs). It is ideal for building Retrieval-Augmented Generation (RAG) pipelines, allowing our chatbot to fetch finance-related data from a knowledge base like finance.txt and generate accurate, context-aware responses.

2. Groq API:

The Groq API offers ultra-fast AI inference for LLMs, enabling near real-time chatbot interactions. In our project, it is used for quick query processing and generating intelligent, finance-focused answers without delay.

3. Pinecone / FAISS (Vector Databases):

These tools are used for efficient document retrieval in RAG systems. They store vector embeddings of financial documents, allowing the chatbot to find and reference relevant content dynamically.

4. Node.js with Express.js:

The backend of the system is built using Node.js and Express.js, providing a robust API layer that connects the React frontend with the Groq AI engine and the RAG pipeline for smooth data flow.

2.2 Limitation of Existing System or Research Gap

1. **Limited Natural Language Understanding:** Many existing chatbot systems struggle with natural language understanding, making it challenging to engage users in meaningful conversations, especially in nuanced or multilingual contexts.
2. **Complex Integration:** Some systems face limitations in integrating with various technology stacks, making it challenging to embed chatbots seamlessly into websites developed using multiple stacks.

3. **Content Generation:** While chatbots can generate responses, they may not always produce creative, contextually relevant content, limiting their ability to provide unique and valuable interactions.
4. **Personalization:** Existing chatbots may fall short in delivering highly personalized experiences, often providing generic responses that do not cater to individual user preferences effectively.
5. **Security and Privacy:** Many chatbots may not be equipped to handle sensitive user data and privacy concerns adequately, posing risks in data protection.

2.3 Mini Project Contribution

To bridge the gap between static content and intelligent learning, this project develops an intelligent Generative AI chatbot integrated with a MERN stack online stock learning platform. Utilizing Prompt Engineering, the chatbot will provide personalized, context-aware assistance to users, offering explanations of financial concepts, guidance on trading strategies, and relevant resource recommendations. This seamless integration enhances user engagement, improves accessibility to educational materials, and establishes a reliable, user-centric environment for stock market education.

Proposed System

3.1 Introduction

This chapter provides a detailed description of the methodology used, the hardware and software components, tools, and screenshots of the developed project.

The website for the **Online Stock Learning Institute** is built using the **MERN stack (MongoDB, Express.js, React.js, and Node.js)**. It integrates a **Generative AI chatbot** powered by **Prompt Engineering** and a **RAG (Retrieval-Augmented Generation)** system utilizing the **Groq API** for fast, context-aware responses.

The foundational front-end technologies are:

1. **MongoDB (Database Layer)**

- Stores user data, chatbot interactions, and learning progress securely.
- Enables flexible document-based storage for finance-related content and RAG retrieval.
- Ensures scalability for growing datasets and real-time query handling.

2. **Express.js (Backend Framework)**

- Serves as the middleware connecting frontend and AI models.
- Handles API routing for chatbot requests, user queries, and Groq API integration.
- Ensures secure and efficient communication between client and server.

3. **React.js (Frontend Framework)**

- Builds an interactive and responsive user interface for the chatbot and stock learning modules.
- Manages real-time conversations, stock data visualization, and dynamic content updates.
- Provides a seamless learning experience with modular components and routing.

4. **Node.js (Runtime Environment)**

- Powers the backend server to execute JavaScript efficiently.
- Manages API calls, RAG processing, and Groq integration with minimal latency.
- Ensures scalability and performance across multiple concurrent chatbot sessions.

Together, these technologies ensure a **responsive, user-friendly, and AI-enhanced learning platform** for stock market education.

3.2 Architecture

MERN Stack (Frontend, Backend, Database):

In our code, we have created a working **online stock learning website** that includes an **integrated AI chatbot** designed to assist users by explaining stock concepts, suggesting study materials, and offering insights based on the user's queries.

React (Frontend):

Our React.js code defines the structure and functionality of the learning platform's user interface.

Here's a brief summary of the code:

1. **Component Structure:** The project follows a modular design using reusable components such as *Navbar*, *CourseCard*, *Dashboard*, and *ChatbotWidget*.
2. **Routing:** We implemented **React Router** for smooth navigation between pages like Home, Courses, Dashboard, and Chatbot.
3. **UI Design:** The layout includes a sidebar for quick navigation, sections displaying trending stocks, and personalized learning modules. The chatbot toggle button allows users to open or close the assistant at any point.
4. **API Integration:** The frontend communicates with the backend via **Axios** for login, signup, chatbot queries, and course updates.
5. **Styling:** We used **Bootstrap** and **CSS3** for layout design and responsiveness, ensuring a professional look and smooth experience across devices.

Node.js and Express.js (Backend):

This part of the system defines the server-side logic and communication between the frontend, database, and AI model.

1. **Server Setup:** The Express.js server handles RESTful API routes for user authentication, chat interactions, and course management.
2. **API Endpoints:** Routes are created for fetching stock-related data, user progress, and chatbot conversations.
3. **RAG Implementation:** When a user asks a question, the backend retrieves the most relevant stock learning materials from the MongoDB database, embeds them using a vector model, and forwards them to the **Groq API** for context-aware generation.
4. **Security:** Middleware like `cors` and `express.json()` ensure secure data transfer and structured

communication between client and server.

5. **Error Handling:** Proper error messages and fallback responses are implemented for failed API calls or empty database results.

Chatbot

The chatbot is the core highlight of our project and enhances user learning with real-time AI support.

1. **User Interaction:** Users can interact with the chatbot through a chat window integrated directly into the React interface.
2. **Input Handling:** The chatbot listens for user queries, appends them to the chat interface, and shows a “Thinking...” message while generating responses.
3. **Retrieval-Augmented Generation (RAG):** The chatbot first retrieves relevant content from the MongoDB database based on user input, then uses the Groq API to generate natural, informative responses combining retrieval and generation.
4. **Dynamic Responses:** For general stock-related questions, the chatbot uses predefined messages or fetches information dynamically via API calls.
5. **Error Handling:** If the API or retrieval process fails, an appropriate error message is displayed, ensuring smooth user experience.
6. **Learning Assistance:** Based on user queries or selected topics (e.g., “How does the stock market work?” or “Explain mutual funds”), the chatbot provides explanations, charts, and study resources tailored to the user’s level.
7. **Visual Feedback:** Messages are color-coded — user inputs appear on the right, and chatbot responses on the left — improving readability and interaction flow.
8. **Integration with Platform:** The chatbot can also recommend which stock module or video to study next, depending on the user’s chat history and progress.

Fig. 3.2.2

3.3 Algorithm and Process Design

Formulating the Problem Statement:

The system aims to design a Generative AI-powered chatbot that assists users in understanding stock markets through interactive, prompt-driven dialogue embedded within a MERN-based website.

Tools / Technology Used:

Frontend: React.js, Tailwind CSS

Backend: Node.js, Express.js

AI Integration: Groq API (LLM), RAG (Retrieval-Augmented Generation)

Editor: Visual Studio Code.

Features Implemented

1. RAG-based Chatbot:

The chatbot is built on a Retrieval-Augmented Generation (RAG) architecture, which enables it to fetch the most relevant data from a document-based knowledge source before generating a response. In this project, the finance.txt document serves as the primary knowledge base containing key concepts of the stock market. When a user asks a query, the chatbot retrieves contextually related information from this document, processes it, and generates an accurate, domain-specific answer. This approach ensures that responses are both precise and educational, avoiding generic AI-generated replies.

2. Prompt Engineering:

Effective prompt engineering has been implemented to ensure that the chatbot provides meaningful, context-aware, and structured responses to user queries. Carefully designed prompt templates help guide the AI model to deliver responses tailored for financial learning. For example, when users ask questions such as “*What is market capitalization?*” or “*Explain the difference between NSE and BSE*”, the system ensures the answers are formatted with clear explanations, examples, and relevant stock market terminology.

3. Stock Information Display:

The platform features a real-time stock information display integrated within the React.js frontend. Using external APIs and dynamic React components, it showcases current market data such as stock prices, daily highs/lows, and percentage changes. This feature helps learners connect theoretical knowledge with live market insights, providing a hands-on understanding of real-world financial trends.

4. AI Query Interface:

The AI-powered query interface allows users to directly interact with the chatbot through a sleek, embedded chat window. Users can type finance-related questions like “*What are mutual funds?*” or “*How does the stock exchange function?*” and receive structured, well-formatted explanations generated via the Groq API. The interface ensures smooth, real-time communication with minimal response delay, enhancing the overall user experience.

5. Modular Codebase:

The entire project is developed using a modular code structure, with separate directories for the frontend (React), backend (Node.js/Express), and documentation (RAG data files). This design promotes better maintainability and scalability, allowing future developers to easily enhance specific components—such as updating the dataset, modifying UI design, or integrating additional AI capabilities—without affecting the rest of the system.

6. Fast AI Inference:

The Groq API is leveraged for ultra-fast inference and low-latency chatbot responses. Its optimized architecture ensures that user queries are processed almost instantly, creating a seamless and responsive chat experience. This is particularly beneficial for educational applications, where learners expect quick and accurate answers while studying stock market concepts.

7. Secure Data Handling:

Security and privacy are maintained through the use of environment variables (.env files) to store sensitive data such as API keys and database credentials. This ensures that no private information is hard-coded into the project files, preventing unauthorized access. The backend also implements secure API handling and follows best practices to maintain data integrity and protect user interactions.

Development and Testing:

Development Steps

1. Setup MERN Environment:

Installed Node.js, MongoDB, and React with all necessary dependencies to establish the full-stack environment. Created a structured folder setup for smooth integration between frontend and backend modules.

2. Frontend Development:

Developed a responsive interface using React components for chatbot interaction and stock visualization. Implemented state management using React hooks for real-time UI updates.

3. Backend Setup:

Built an Express.js server to handle API requests, process user input, and connect with the Groq API. Integrated the RAG pipeline to retrieve relevant context and generate intelligent responses.

4. Knowledge Base Creation:

Prepared a finance.txt file containing structured data on stock markets, NSE/BSE, and investment terms. This file supports the RAG model in providing domain-specific, factual answers.

5. Groq API Integration:

Connected the backend with the Groq API for fast, low-latency AI response generation. Used prompt-based architecture to enhance the chatbot's understanding and accuracy.

6. Styling & Responsiveness:

Used Tailwind CSS to design a clean and responsive interface across devices. Ensured proper alignment, readability, and accessibility throughout the website.

Testing Steps

1. Functionality Testing:

Verified chatbot responses for accuracy, clarity, and relevance to finance-related queries. Ensured all core features, including RAG retrieval, worked as intended.

2. API Testing:

Tested backend routes using Postman to confirm correct data exchange between React and Node. Ensured consistent and error-free API communication.

3. Cross-Browser Testing:

Checked compatibility and performance on Chrome, Edge, and Firefox browsers. Ensured responsive design and layout consistency across all platforms.

4. Performance Evaluation:

Measured chatbot response times using Groq's inference metrics. Optimized API calls for faster and smoother interaction.

5. User Interaction Testing:

Simulated multiple finance-related conversations to test contextual understanding. Evaluated chatbot flow, message clarity, and UI responsiveness.

6. Error Handling:

Validated fallback mechanisms for failed API requests. Displayed appropriate user-friendly error messages during network or API errors.

7. Security Testing:

Checked .env file protection for API key confidentiality. Ensured all sensitive data remained secure during API communication.

3.4 Details of Hardware & Software:

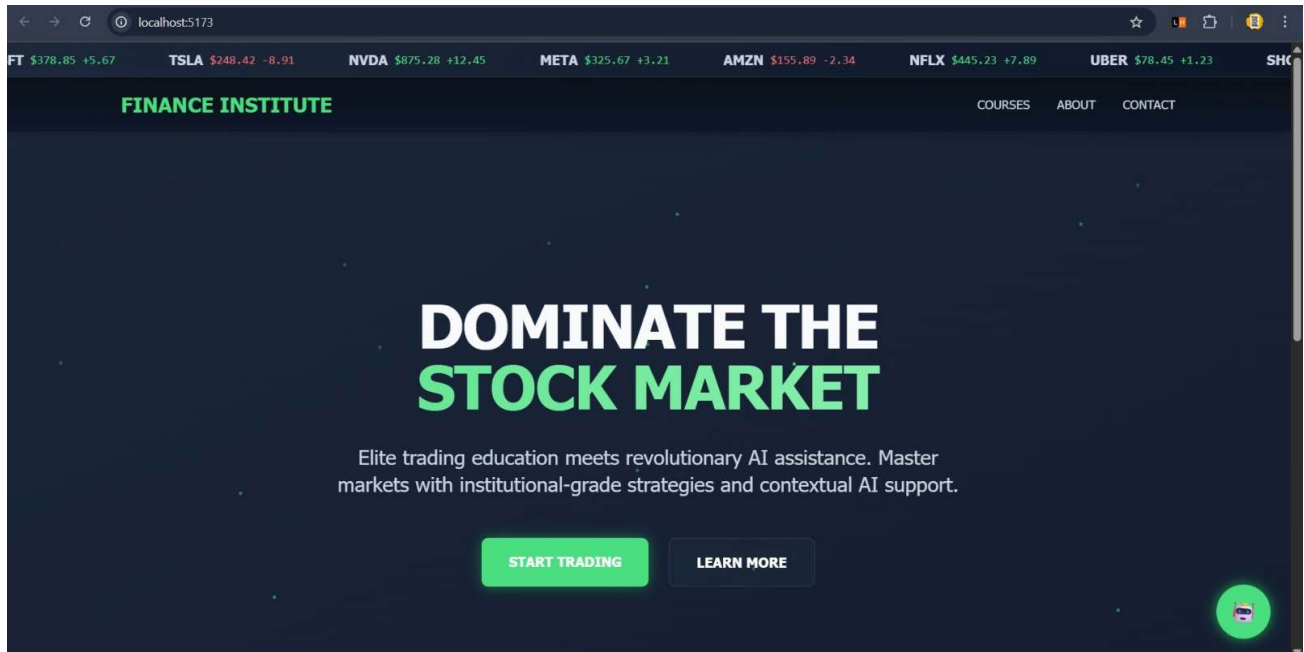
Hardware Configurations :

- Microsoft Windows 11
- Processor: Intel Core i5 10th gen
- System type : x64-bit
- Physical memory : 8 GB

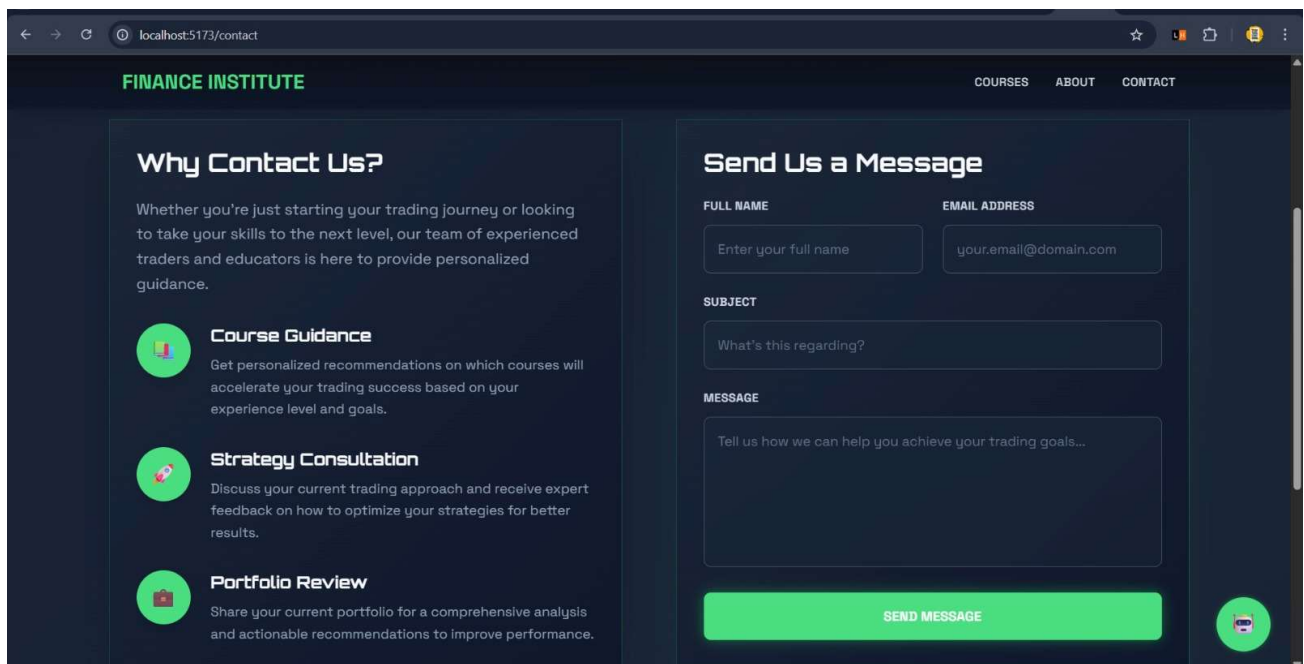
Software Configurations:

- Visual Studio Code 1.72.2
- HTML5
- CSS3
- ECMAScript 2022 (ES13)

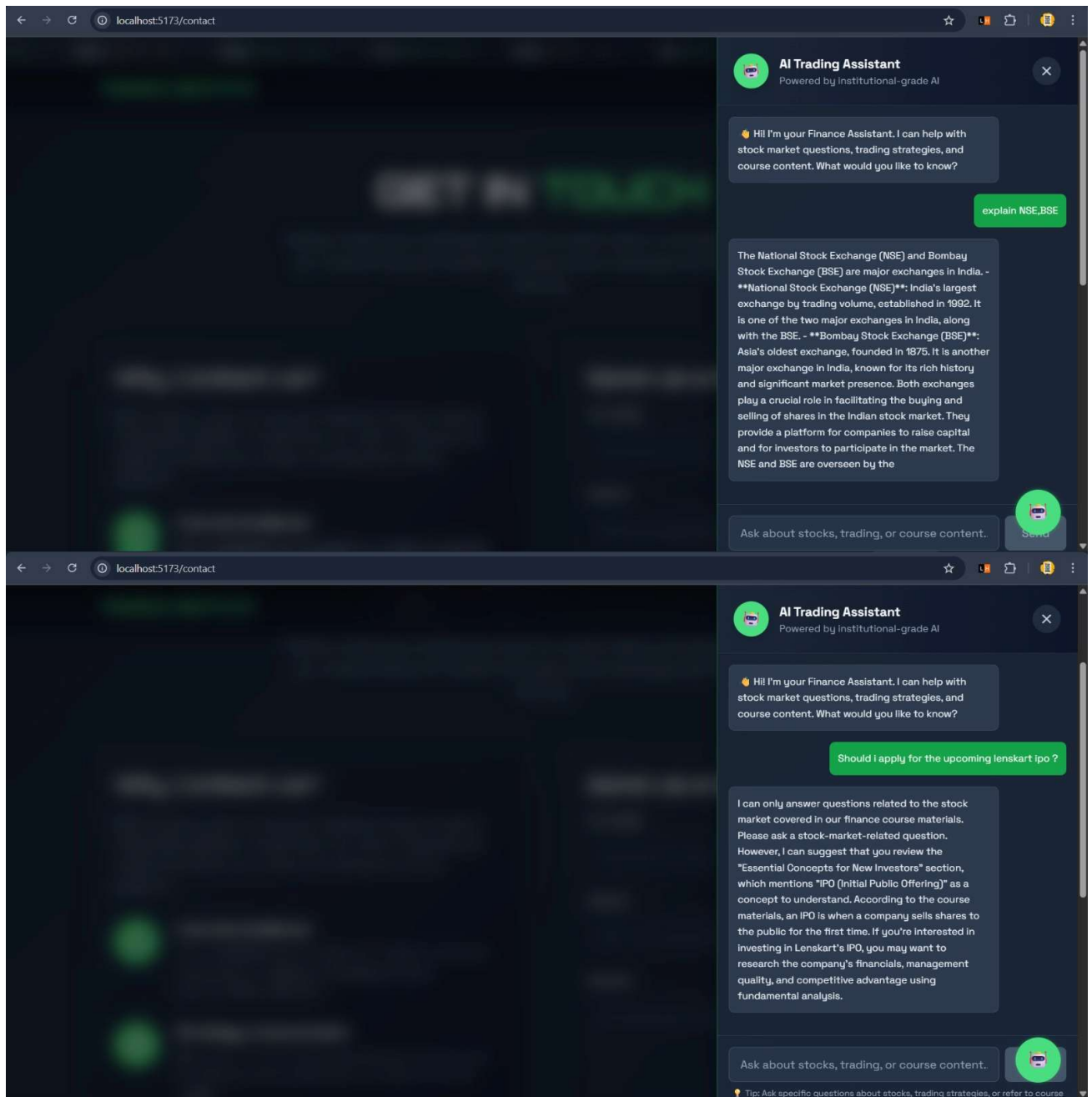
3.5 Results:



(Fig. 3.5.1 – Main Page)



(Fig. 3.5.2 Contact us)



(Fig. 3.5.3 – ChatBot)

3.6 Conclusion:

The above concept describes the development of an advanced Generative AI-powered chatbot designed for seamless integration into a MERN-based online stock learning website. This chatbot utilizes the capabilities of Prompt Engineering and AI models to provide intelligent, context-aware, and personalized interactions for learners. By embedding it within the educational platform, users can engage with the chatbot to understand stock market concepts, receive course recommendations, clarify doubts, and access real-time examples related to trading and investments. This integration aims to make the learning process more interactive, adaptive, and efficient, ultimately enhancing the overall user experience on the stock learning website.

References

<https://chat.openai.com/>

https://www.researchgate.net/publication/381461839_Retrieval_Augmented_Generation_RAG_based_Restaurant_Chatbot_with_AI_Testability

<https://www.mdpi.com/2076-3417/15/8/4234>